

Agentic Computer Vision Project Proposals — Laptop Camera Only

For a 4–6 person undergrad AI/ML team | Hardware: RTX-class gaming laptops, no external sensors

Design philosophy, unchanged: every project pairs a *fast, local* CV model (does the seeing, runs on your GPU in real time) with a *slow, cloud* reasoning model like Claude (does the thinking/deciding), connected by an agent loop. Dropping the ESP32 actually sharpens the scope — everything now has to work end-to-end through a single laptop webcam, which pushes you toward smarter software design instead of extra hardware to compensate.

Project 1: SousChef — The Real-Time Cooking Companion

1. Architecture Overview

Prop the laptop up facing the counter/stove. The system watches the cooking process step-by-step, tracks which recipe steps have actually been completed (ingredient added, pan on heat, etc.), and an agent narrates guidance, catches mistakes ("that looks like it's browning too fast"), and adapts the recipe in real time if it sees a substitution or a skipped step.

```
[Laptop webcam, counter-facing] --> [Frame sampler, ~2-4 fps]
                                     |
                                     [Local detector: ingredients, utensils,
                                     pan/stove state (YOLOv8 + open-vocab
                                     detector for unlisted ingredients)]
                                     |
                                     [Step tracker: compares detected state
                                     to expected recipe step sequence]
                                     |
                                     [Agent (Claude, vision-capable): only
                                     called on step transitions or anomalies –
                                     not every frame, to control cost/latency]
                                     |
                                     [Outputs: spoken/on-screen next-step guidance,
                                     real-time correction, recipe re-planning if
                                     a substitution or mistake is detected]
```

2. Societal ROI & Ubiquitous Impact

Cooking mistakes — wrong order of operations, missed steps, over/undercooking — are a universal daily friction point, especially for students who are new to cooking and don't have someone to check their work. Static recipe apps can't see what you're actually doing; this can. Target demographic: students and young adults living independently for the first time

— directly testable on your own team's kitchens. The ROI is turning a passive recipe card into an active collaborator that only speaks up when it's actually useful, not step-by-step hand-holding that gets ignored.

3. Novel Tech & Agentic CV Stack

- **Detection:** YOLOv8/v9 for common kitchen objects + **Grounding DINO or OWL-ViT** (open-vocabulary) so it can flag ingredients it wasn't explicitly trained on — a real step up from closed-set classification most CV courses stop at.
- **State tracking, not just detection:** the interesting new skill here is building a **step-sequence tracker** — a lightweight state machine that reconciles noisy frame-by-frame detections into "has step 3 actually happened yet?" This is a genuinely useful pattern for any process-monitoring CV system, not just cooking.
- **Vision-capable agent calls:** send Claude actual cropped frames (not just labels) at decision points, using **multimodal prompting** — worth explicitly comparing "text-only state description" vs. "send the frame" in your report, since it's a real architectural tradeoff (cost/latency vs. accuracy).
- **Cost control pattern:** call the LLM only on state transitions/anomalies, not every frame — a core lesson in agentic systems that most tutorials skip (naively calling an LLM on every frame is slow and expensive; this is the fix).
- **Voice output:** local TTS (e.g., Coqui or even browser Web Speech API) so guidance is spoken hands-free while cooking — small but very real UX addition.

4. Team-Based Implementation & AI Collaboration

Role	Owns	Delegate to AI coding agent	Architect manually
CV Engineer	Detection pipeline, frame sampling	ONNX export, augmentation scripts	Class list, confidence thresholds, when to trust open-vocab vs. closed-set detector
State/Logic Engineer	Step-sequence tracker	Boilerplate state machine library setup	The actual reconciliation logic — how much noisy-detection disagreement before you trust a transition
Agent Engineer	Claude multimodal tool-use, prompt design	API plumbing, retry/error handling	When to call the agent at all (the cost-control policy is your core contribution)
UX Engineer	Voice output, on-screen overlay	TTS integration boilerplate	Interruption/timing design — guidance has to land at the right moment or it's useless

Sequencing: Week 1–2 get open-vocab ingredient detection stable on live video. Week 3 build the step-tracker state machine on a fixed 3-step recipe. Week 4–5 wire in the Claude agent calls at transitions. Week 6+ test on 3–4 real recipes cooked live by teammates — real cooking sessions are your best demo footage.

5. Deployment & Scalability

MVP runs entirely on one laptop. Package it as a lightweight desktop app (Electron front end + local Python inference server) so any student can run it on their own machine pointed at their own counter — no cloud dependency for the CV layer, only for the periodic Claude calls, which you can cache/batch to control API cost. Scale path: swap the local detector for a browser-side ONNX.js model and it becomes a pure web app, no install required.

Project 2: ErgoGuard — The Agentic Posture & Focus Coach

1. Architecture Overview

The laptop's own webcam continuously tracks posture and screen distance during study/work sessions. Instead of naive "you're slouching" alerts (ignored within a day), an agent reasons over *patterns* — session length, posture trend over time, time of day — and decides when and how to intervene, escalating from a subtle on-screen cue to a specific, personalized coaching message.

```
[Laptop webcam] --> [MediaPipe/BlazePose landmarks, real-time, local]
                        |
                        [Posture feature extractor: neck angle, shoulder
                        symmetry, distance-to-screen, blink rate]
                        |
                        [Rolling session buffer, 30–60s windows]
                        |
                        [Agent decides: ignore / subtle cue / full coaching moment]
                        |                                     |
                        [On-screen ambient indicator]         [Claude: generates a
                        – dimming corner icon,                 specific, context-aware
                        unobtrusive color shift]                tip referencing the
                                                                actual session
pattern]
```

2. Societal ROI & Ubiquitous Impact

Musculoskeletal strain and eye fatigue from long screen sessions is one of the most common, least-addressed health issues among students — existing posture apps have near-zero adherence because they nag identically regardless of context. Target demographic: your entire team and every classmate pulling long study/coding sessions, making this

trivially easy to pilot on real users. The genuine ROI is behavioral: an agent that adapts *how and when* it intervenes based on actual observed patterns is a meaningfully better product than a fixed-threshold nag.

3. Novel Tech & Agentic CV Stack

- **Pose estimation:** MediaPipe Pose, or **RTMPose** for a stretch target (faster and more accurate, less commonly taught) — real-time on your GPU with room to spare.
- **Temporal reasoning:** the core new skill — instead of frame-by-frame classification, you build a sliding-window feature aggregator feeding a lightweight decision policy. Worth exploring a small state machine, or even a tiny fine-tuned classifier for "escalate vs. don't."
- **Agent layer:** Claude generates coaching messages conditioned on *structured session summaries* — send it trend numbers, not raw video, both for speed and for privacy (a design decision worth calling out explicitly in your report).
- **Privacy-first architecture:** all pose data and video stay entirely local; only anonymized session summaries (e.g., "42 min session, posture score declining last 10 min") ever leave the machine. This is a legitimately good architectural talking point, not just a nice-to-have.
- **Delivery:** a small always-on-top overlay widget (Electron or a native menu-bar app) rather than a full window — the interaction design is part of the technical challenge here.

4. Team-Based Implementation & AI Collaboration

- **CV engineer** owns the pose pipeline and posture-metric definitions — architect the angle/metric thresholds yourself (this is domain-specific and where your grade differentiates from a tutorial clone); delegate MediaPipe integration boilerplate to an AI coding agent.
- **Agent engineer** owns the escalation decision policy and Claude prompt design — this is the hardest, most manual piece; don't let an AI agent design the intervention psychology, but do delegate the API plumbing.
- **UX/frontend engineer** owns the overlay widget — delegate component scaffolding, architect the escalation visuals (subtlety is the whole design problem) manually.
- **Data/eval engineer** — runs a real usability pilot on 2–3 teammates over a week and logs adherence/annoyance — this genuinely differentiates the project from a toy demo.

5. Deployment & Scalability

Ships naturally as a lightweight background app — no cloud dependency for the CV layer at all, only for coaching-message generation, which can be cached and rate-limited to control API cost. Scale path: package as a real installable app (Electron, or a Python system-tray

app) and distribute to classmates as a genuine pilot user base — you'll get real adherence data most class projects never have.

Project 3: SignBridge — Real-Time Sign Language to Fluent Captioning

1. Architecture Overview

The laptop webcam captures a signer. A local model recognizes sign gestures/sequences from hand, face, and body landmarks; because raw gesture recognition alone produces choppy, grammatically broken output, an LLM agent sits downstream to reconstruct fluent, context-correct sentences from the recognized gesture stream in real time.

```
[Laptop webcam] --> [MediaPipe Holistic: hand + pose + face landmarks]
                        |
                        [Sign sequence classifier: temporal model
                        (ST-GCN / transformer over landmark sequences)]
                        |
                        [Rolling gloss buffer, e.g. "I GO STORE YESTERDAY"]
                        |
                        [Claude: reconstructs fluent caption - "I went to
                        the store yesterday" - using conversational
                        context/history across turns]
                        |
                        [Live caption overlay rendered on the same screen,
                        or streamed to a second device via WebRTC]
```

2. Societal ROI & Ubiquitous Impact

Real-time communication access for Deaf/hard-of-hearing individuals remains a major daily friction point — professional interpreters aren't always available, and most gesture-recognition demos stop at stilted "glossed" output. This is your most explicitly accessibility-driven project, with a clear target demographic (Deaf/HoH students, and hearing people interacting with them) where even an imperfect but *fluent-sounding* system has real utility. It's also the hardest to get accuracy right on given limited training data for regional sign variants — worth being upfront with your team about that tradeoff.

3. Novel Tech & Agentic CV Stack

- **Landmark extraction:** MediaPipe Holistic (hands + pose + face) — richer than typical hand-only pipelines, needed because sign grammar uses facial expression and body position, not just hand shape (a genuinely non-obvious CV insight worth foregrounding in your writeup).
- **Temporal sign classifier:** a small ST-GCN or transformer over landmark sequences, trained on a public dataset like WLASL — a real step up from static-image classification.

- **The agentic novelty:** using an LLM not for vision but as a **grammar/fluency reconstruction layer** over noisy structured output — this pattern (raw model output → LLM cleanup/reasoning layer) generalizes to many other CV problems and is worth calling out as your core technical contribution.
- **Streaming (optional stretch):** WebRTC to send the live caption feed to a second laptop/device in the room, useful if the signer and reader aren't looking at the same screen — entirely software, no extra hardware needed.

4. Team-Based Implementation & AI Collaboration

Given the difficulty, split by pipeline stage rather than by role:

- **Data/model team (2 people):** curate a workable subset of WLASL or similar, train the ST-GCN classifier — delegate data loader and augmentation boilerplate to AI agents, but architect the label taxonomy and train/val split yourselves (data leakage here is an easy, grade-costing mistake).
- **Agent/NLP person:** prompt design for gloss-to-fluent-caption reconstruction, maintaining conversational context across turns — core manual work, this is your differentiator.
- **Frontend person:** caption overlay UI and optional second-device streaming — delegate scaffolding, architect the caption timing/buffering yourself.

Honest scoping note: consider limiting your MVP to a **closed vocabulary of ~50-100 common signs** rather than open-domain ASL — this keeps the CV problem tractable while still fully exercising the interesting agentic reconstruction layer.

5. Deployment & Scalability

MVP is a single laptop app: webcam in, captions overlaid on the same screen. Path to real deployment: package the classifier as a browser-side ONNX.js model so it runs entirely client-side with no install, and host the reconstruction agent behind a lightweight API — removing the local-Python dependency entirely, which is the real long-term accessibility win (anyone opens a web page, no setup).

Project 4: WellCheck — The Home Wellness & Fall-Detection Guardian

1. Architecture Overview

A single laptop webcam, propped in a common area (living room, elderly relative's home, dorm common room), continuously monitors for falls, prolonged inactivity, or distress signals — using local pose/motion analysis for speed and privacy, with an agent handling the *triage* decision (false alarm vs. real emergency) and drafting a clear, calm alert message to a designated contact, rather than blasting sirens on every twitch.

```
[Laptop webcam, fixed position] --> [Local pose estimation, real-time]
```

```
|
```

```
[Motion/fall feature extractor: sudden  
vertical drop, prolonged floor-level  
pose, absence of movement over time]
```

```
|
```

```
[Local anomaly classifier: flags candidate  
events, filters obvious non-events]
```

```
|
```

```
[Claude: triages severity using recent  
context – was there a fall pattern, or a  
normal sit-down? drafts a plain-language  
alert only if warranted]
```

```
|
```

```
[Notification to a designated contact +  
auto-logged, timestamped event summary]
```

2. Societal ROI & Ubiquitous Impact

Fall detection and wellness monitoring for elderly individuals or people living alone is a genuine, high-stakes daily-life problem — commercial solutions exist but are often expensive wearables people forget to wear, or dedicated cameras that are costly to install. A laptop-camera-based system is a low-cost, easy-to-pilot proof of concept for a real need. Target demographic: elderly relatives living alone, or anyone with a health condition that carries fall risk. The ROI is specifically in the triage layer — alarm fatigue is the actual reason most monitoring systems get disabled by users; an agent that only escalates genuinely concerning events, and explains *why* in plain language, is a meaningfully more trustworthy system.

3. Novel Tech & Agentic CV Stack

- **Pose-based fall detection:** MediaPipe Pose or a lightweight action-recognition model tracking rapid vertical center-of-mass drop and post-fall stillness — a well-studied but genuinely nontrivial CV problem (distinguishing "sat down on the floor to stretch" from "fell" is the real challenge, not the pose estimation itself).
- **Local-first privacy design:** all video processing happens on-device; only structured event summaries (never raw frames, never continuous footage) are sent to the cloud agent — an important, explicitly documented design decision given the sensitivity of home monitoring.
- **Agent layer:** Claude with tool-use — `get_recent_activity_pattern()`, `send_alert()`, `log_event()` — deciding whether an event needs escalation and drafting the actual notification text. A good discussion point for your report: where do you draw the line between rules-based logic and LLM judgment for a safety-critical decision like this?

- **Ethics/scoping note to bake into your writeup:** explicitly restrict this to fall/inactivity detection only — no facial recognition, no identity tracking, no continuous recording storage. State this clearly; that restraint is itself a demonstrable, gradeable design decision, especially important for anything framed around monitoring people in their homes.

4. Team-Based Implementation & AI Collaboration

- **CV engineer:** pose pipeline and fall/inactivity feature design — architect the fall-vs-normal-movement heuristics yourself (this is the crux of the whole project); delegate MediaPipe integration boilerplate to an AI agent.
- **Agent engineer:** severity triage logic and Claude tool-use design — the escalation policy is your core intellectual contribution, don't outsource the *policy*, only the surrounding API code.
- **Data engineer:** stage a labeled dataset of real falls vs. normal movements (with consenting teammates, safely on padding/mats) — a legitimate, careful data-collection exercise; delegate the labeling-tool setup, own the collection protocol and safety plan yourself.
- **PM/ethics lead:** explicitly owns the privacy/scoping documentation — meaningfully improves your evaluation for a project touching home monitoring, not just a checkbox.

5. Deployment & Scalability

MVP runs on one laptop watching one room, with alerts sent to a phone via a simple notification service (email/SMS API, or a push notification service). Scale path: package as an installable app for a dedicated always-on device (an old laptop repurposed as a fixed monitor is a realistic, low-cost real-world deployment), with the triage agent logic hosted centrally if you ever wanted to support multiple households — the architecture doesn't need to change shape to scale, only the deployment target.

Cross-Project Recommendation

SousChef and **ErgoGuard** are the most achievable within a semester while still fully exercising open-vocabulary detection or temporal pose reasoning, plus real agentic tool-use with Claude — and both are trivially easy to pilot on your own team. **SignBridge** has the highest social-impact ceiling but the hardest data problem — take it on only if someone's comfortable owning model-training risk. **WellCheck** is the most "real product" of the four in terms of societal stakes, but requires the most careful, explicit ethical scoping given it's a home-monitoring system — worth it if your team wants that discussion to be part of the deliverable, not just the code.